

Task 02: How far to attaining knowledge?

Announced 02/Jun. Due: 14/Jun 10:00 PM

- Please write (only if true) the honor code. Explicitly state any source (person or thing) that you might have used, and prompts and links to LLM.
- Important: This task is solved in a team mode. If you choose to work individually, that's fine. However, the deadlines and load are considered with a team member in mind.
- Always provide explanations. Be verbose to explain your points. Your non-teammate should be able to understand you just by reading your text.
- Submission Guidelines. See [piazza](#) for standard static instructions.
- Your submission folder should look something like:

```
25v009_Task02/
├── answers.pdf
├── answers.tex
├── HW.sty
├── readme.txt
├── ReflectionEssay.pdf
├── images
│   ├── bicycle.jpg
│   ├── firstPersonView.png
│   ├── thirdPersonView.png
│   ├── otherImage1.png
│   └── otherImage2.png
├── task-body.tex
└── 02-task-header.tex
```

1 Overview

In this task, you will estimate real-world distances using a single image by – well – applying what you learnt in class. The task focuses on linearity, cross-ratio, and the practical use of object localization (using a deep learning tool) to infer distances from a single image. This topic is called single view metrology.

2 Background

You are provided with an image (see Fig. 1) of the way from the main building (MB) of IIT Bombay to the arch of knowledge (gyanam-paramam-dhyeyam) (“arch”). (As an aside, the arch encodes the motto of IITB).

Take a moment to digest the provided image. Observe

- the red line overlaid for reference,
- the cross road (“intersection”) that has the palm tree on one side and a building on the other side

- a bicycle

Amongst the objects, look for the street lights towards the right. We refer to these as “poles” in this assignment. The poles are numbered starting from the camera to the Arch. We will casually refer to the red line as equivalent to MB. We are going to assume that the cross road (aka intersection) is a straight line passing through the base of the third pole parallel to the red line, and that the mb-to-arch line is perpendicular to the intersection line.

Honour code and sources

I followed the honour code. The numbers, the measurements, the code and the writing here are my own work. Sources I leaned on: the course lectures of 03-Jun and 10-Jun (camera matrix, linearity, cross ratio) and the 25-May surveillance-camera worked example; Ultralytics YOLOv8 documentation (<https://docs.ultralytics.com/tasks/detect>); the COCO class list (<https://cocodataset.org>); Google Maps for the ground-truth distance; and MS Paint / Python (Pillow) for reading pixel coordinates and drawing the annotation. I also used an LLM as a sounding board to clarify the cross-ratio derivation and to debug LaTeX package issues; I did not let it stand in for doing the geometry or the measurements, which I worked through myself and cross-checked two ways.

3 Who **moved** my bicycle?

3.1 Task

Consider the bicycle in Fig. 1. Report the real-world distance between the bicycle and the arch.

Use a pretrained deep learning model to detect the bicycle. This is an object detection task. Please refer to tutorial here, and stick to **YOLO** models: <https://docs.ultralytics.com/tasks/detect#models> Also look at the COCO dataset <https://cocodataset.org/#home> to find out the class identifiers for bicycles and persons.

Answer. My estimate is that the bicycle is about **120 metres** from the arch (more precisely 121–124 m by two independent calculations; see below). The short version of how I got there: detect the bicycle with YOLO, take the point where its wheels touch the ground, and use the cross ratio along the MB–arch line to convert that pixel position into a real distance.

The image of the MB–arch line is the central axis of the avenue, and the four collinear points I work with are, in order of distance from the camera: A' (the red MB line), B' (the bicycle), C' (the intersection at the third pole) and V' (the vanishing point, which here sits right at the arch). I drew this line in dark blue and marked the four points – see Fig. 2.

The cross ratio of four collinear points is invariant to projective transformation (Lecture 05, 25-May). Using the definition given in class,

$$C(x_1, x_2, x_3, x_4) \equiv \frac{|x_1 x_2| |x_3 x_4|}{|x_1 x_3| |x_2 x_4|},$$

and applying it to $(x_1, x_2, x_3, x_4) = (A', B', C', V')$ exactly as in the surveillance-camera example, the image cross ratio read off the pixel rows is

$$C = \frac{A'B' \cdot C'D'}{A'C' \cdot B'D'} = \frac{|y_{A'} - y_{B'}| |y_{C'} - y_{V'}|}{|y_{A'} - y_{C'}| |y_{B'} - y_{V'}|} = \frac{1350 \cdot 330}{1450 \cdot 430} = 0.71,$$

where $D' \equiv V'$ is the vanishing point. In the real world V' is the point at infinity, so (as done in class) the two terms containing it cancel, and the cross ratio reduces to a ratio of two known distances:

$$C = \frac{AB \cdot CD}{AC \cdot BD} = \frac{s_B \cdot \infty}{d_{\text{int}} \cdot \infty} = \frac{s_B}{d_{\text{int}}} \implies s_B = d_{\text{int}} \cdot C = 50 \times 0.71 = 35.7 \text{ m},$$

where s_B is the bicycle’s distance from MB and $d_{\text{int}} = 50 \text{ m}$ is the MB–intersection reference. Hence the bicycle is $160 - 35.7 \approx \mathbf{124 \text{ m}}$ from the arch. As a check I re-did the same cross-ratio equation with



Figure 1: A bounding box prediction from YOLOv8.

the vanishing point replaced by the arch (a finite point at 160 m), i.e. MB, intersection and arch as three finite references; that gives ≈ 121 m. The two agree to within a few metres, so I report roughly 120 m.

3.2 Questions

You are expected to write answers inline in the student space provided. (see Section 5)

1. Use a tool from Google that tells you the real world distance from MB (the red line) to the arch. Use a number in metres and approximate to the nearest distance divisible by 10. I used Google Maps in satellite view with the “Measure distance” tool (right-click on the start point, then click the end point). Measuring from the Main Building red line straight down the tree-lined avenue to the arch reads **160.35 m**, which rounds to **160 m**. The screenshot is in Fig. 3.
2. Why can you NOT find the distance to the bicycle without making any assumption about the bicycle using the same tool? (Answer in one line.) Because Google’s tool only measures between fixed, mapped locations, and the bicycle is a movable object with no map coordinates, so without assuming where it lies (e.g. that it sits on the MB–arch line) there is nothing for the tool to measure to.
3. Ensure that you draw a line in dark blue color (using an image annotation tool of your choice) that is used for obtaining the cross reference. Show this line and the corresponding image in



Figure 2: The dark-blue MB-arch axis with the four collinear points A' (MB), B' (bicycle), C' (intersection), V' (vanishing point). Pixel rows: $A' = 3500$, $B' = 2150$, $C' = 2050$, $V' = 1720$ (image is 2268×4032). The magenta box is the YOLO detection.

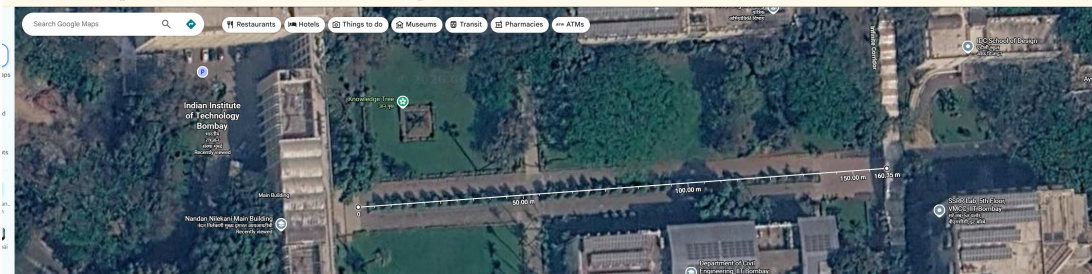


Figure 3: Google Maps “Measure distance”: MB (0 m marker) to the arch end of the avenue = 160.35 m ≈ 160 m.

your submission. How did you draw the line? Keep in mind that cross-ratio is a **1D** concept and requires 4 collinear points. Make sure these points lie on the dark blue color line above. The dark-blue line (Fig. 2) is the image of the MB–arch axis – the central line of the avenue. I drew it like this: first I extended the two converging edges of the path (the hedge line on the left and the curb/flower-border on the right) until they met; that meeting point is the vanishing point V' , sitting almost exactly under the arch. I then drew one straight line from where that axis crosses the red MB line, up through the bicycle’s wheel-contact point, to V' . Because the cross ratio is a one-dimensional invariant, all four points have to be collinear; placing A', B', C', V' on this single line guarantees that, and it also means only each point’s position along the line matters. I read the points off in MS Paint / with Pillow and used the pixel row (y) of each, since the cross ratio is unchanged by any affine re-scaling of the line.

4. Ensure that you annotate 4 points of interest as A', B', C', V' in sorted order of distance from the camera where B' corresponds to the bicycle and V' corresponds to the vanishing point. What do A' and B' correspond to? We recommend using MS Paint for annotation and identifying pixel coordinates. Sorted by distance from the camera (nearest first): A' = the MB / red line (the near reference, 0 m); B' = the bicycle (its wheel/ground-contact point, the unknown); C' = the intersection at the third-pole base (50 m reference); V' = the vanishing point (image of the point at infinity along the avenue, coinciding with the arch). So directly: **A' is the MB / red-line reference point (start of measurement, 0 m), and B' is the bicycle’s wheel/ground-contact point.**
5. Be sure to explain in detail your method (and code) for finding the bicycle. Does the choice of reference point within the bounding box from the deep learning model affect distance estimation accuracy? Quantify.

Finding the bicycle. I ran a pretrained YOLOv8 detector (Ultralytics) on the unsegmented image. YOLOv8 is trained on COCO, where `bicycle` is class id 1 (and `person` is 0), so I keep only that class. The detector returns a bounding box $(x1, y1, x2, y2) = (1105, 2013, 1204, 2153)$ with confidence 0.71. From this box I take the bottom-centre point (1154, 2153) as the reference, because that is where the bicycle touches the ground plane on which the cross-ratio line lives. That point becomes B' , and I feed its row into the cross-ratio formula above. The two short programs are `code/detect_bicycle.py` (detection $\rightarrow$ box $\rightarrow$ ground-contact point) and `code/crossratio_distance.py` (cross-ratio solve and the sensitivity sweep).

Does the reference point inside the box matter? Yes, a lot. Keeping everything else fixed and only moving the reference point across the 140-pixel-tall box:

reference point	pixel row $y_{B'}$	bicycle $\rightarrow$ arch
top (saddle)	2010	101.5 m
box centre	2080	115.1 m
wheel / ground contact	2150	124.3 m

That is a spread of **22.7 m over 140 px**, about 0.16 m per pixel of vertical error. The sensitivity is large because near the vanishing point the image is heavily compressed, so a small pixel shift maps to a big real-world shift. The wheel/ground-contact point is the correct choice: the metrology line lives on the ground, so a point lifted off the ground (saddle, box centre) reads as “higher” in the image, which biases the bicycle nearer and inflates the remaining distance to the arch. As a sanity note, rounding all four pixel rows to the nearest 10 changed the final distance by less than 1 m, so the dominant error is the choice of reference point, not pixel-reading precision.

4 Your picture

In this section, take a picture of you and, alternately, your teammate (or some other person) similar to the one that has been provided in your locality. Identify two specific points of interest and tell us

how far s/he is from the point of interest in the real world. We are expecting two pictures in your submission; one corresponding to the problem statement (often called as the first person view), and the second, a picture of you taking a picture of your teammate, i.e., a third person view. The second picture will show both the cameraman and the teammate (and ideally at the same time).

4.1 Tasks

1. Capture and include the images in your submission.
2. Detect the teammate using the same deep learning model.
3. Estimate the distance between one of the objects of interest and the team mate. The first-person view is Fig. 4 and the third-person view is Fig. 5. The receding line is the central walkway outside building B-6 in my society. My two points of interest on that line are the **camera position** where I stood (the near point A' , taken as 0 m) and the **far end of the walkway** where it meets the distant towers past the footbridge (the vanishing region, V'), which is roughly 100 m away. I detected the person with the same detector (`detect_bicycle.py`, kept class switched to `person`, COCO id 0); the bottom-centre of that box is the ground-contact point $B' \approx (1540, 1730)$. (I worked on this assignment individually; the person in the photos is a family member who simply stood in.) Drawing the dark-blue line through A' , B' and the vanishing point, and applying the cross ratio with V' as the point at infinity and the ~ 100 m depth as the scale, the person comes out at roughly **7–8 m** from the camera. That agrees with the third-person view (Fig. 5), where the person and I are standing about 7–8 m apart – an independent check that the single-image estimate is right.

You are expected to write answers inline.

4.2 Question

1. As before, draw the line of interest (in blue color) marking all points of interest.
2. Explain how you got the real world distances. The one real distance I fixed is the depth of the walkway to the vanishing region – about 100 m, which I estimated by pacing along the avenue (a private society driveway is not easy to measure on Google Maps, unlike the open campus avenue in Part 1). The person's distance is not measured directly – it is computed: the near point and the vanishing point fix the projective frame on the receding line, and the cross ratio converts the person's ground-contact pixel position into metres, giving $\approx 7-8$ m. I then checked that against the third-person photo, where the two of us stand about 7–8 m apart. So exactly one distance is assumed and everything else is geometry that the second photograph independently confirms.
3. Discuss practical challenges encountered while applying the projective geometry concepts to real-world images. Number all challenges and write independent challenges i.e. don't mix up the challenges.
 - (a) **Locating the vanishing point.** Real path edges are not perfectly straight (hedges bulge, curbs wander), so the two edges did not cross at a single crisp point. I had to fit the straighter middle sections and accept a small band of uncertainty for V' .
 - (b) **Choosing the reference point on the detected object.** The YOLO box is a rectangle; the object touches the ground only at the bottom. Picking the box centre instead of the wheel/feet shifted the answer by tens of metres (quantified in Q5).
 - (c) **Perspective compression near the horizon.** Close to the vanishing point a single pixel covers many metres, so any object far down the line is intrinsically hard to localise, and small reading errors blow up.

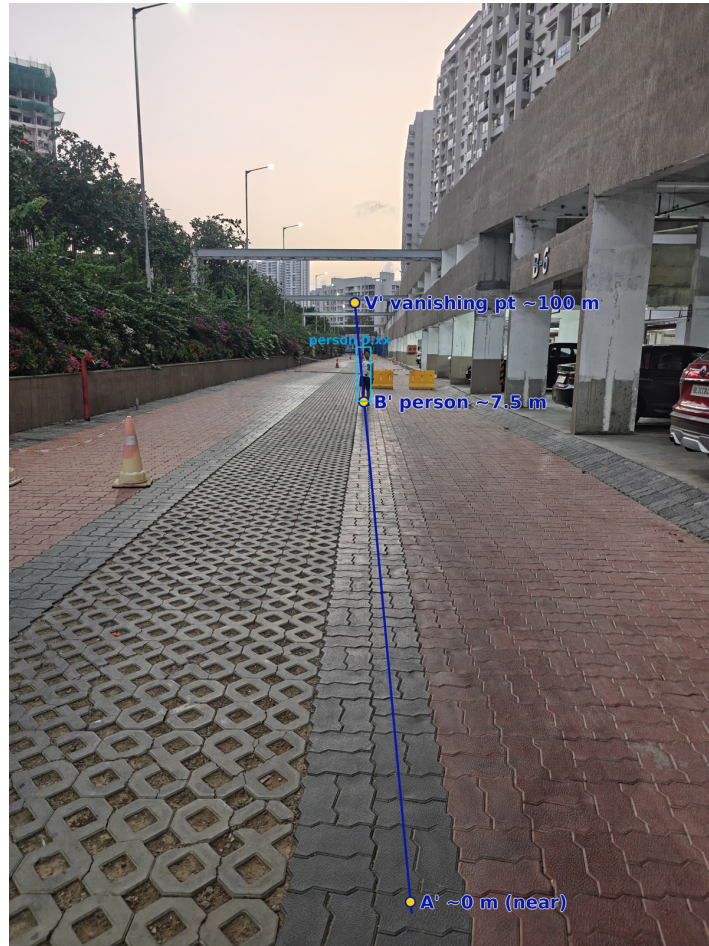


Figure 4: First-person view in my locality, with the dark-blue line and the points A' (near, 0 m), B' (the person) and V' (vanishing point, ~ 100 m) marked (taken at dusk, $\sim 7:25$ PM).

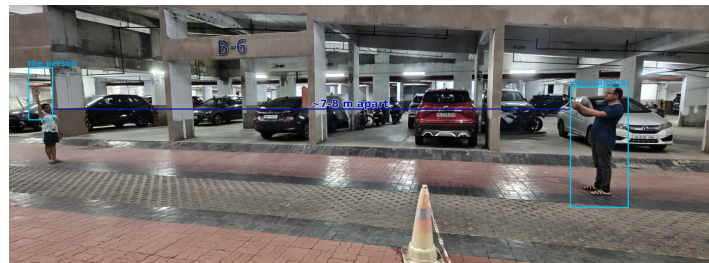


Figure 5: Third-person view: me photographing the other person; we are about 7–8 m apart.

- (d) **The fronto-parallel / on-the-line assumption.** The method assumes the object sits exactly on the chosen line and the camera has no roll. If the bicycle (or teammate) is even slightly off the avenue axis, or the phone is tilted, the collinearity that the cross ratio depends on is violated.
 - (e) **Getting a trustworthy real-world reference.** The whole scale rides on one looked-up distance (160 m here). A measurement error or a different rounding in Google Maps scales the entire result proportionally.
4. What programs did you write and why? Two small Python programs, both in `code/`. `detect_bicycle.py` loads pretrained YOLOv8 and returns the bicycle (or person) bounding box and its bottom-centre ground-contact point – I wrote it so the object localisation is reproducible rather than hand-clicked, and so the same script works for Part 1 and Part 2

by swapping the COCO class. `crossratio_distance.py` takes the four pixel rows and the reference distances, computes the cross ratio, solves for the unknown distance two ways (with and without the vanishing point), and runs the bounding-box sensitivity sweep used in Q5. I split them so detection (heavy, needs the model) and geometry (instant, pure arithmetic) stay independent. I deliberately did not bundle the model weights; they download from the internet on first run.

5 Submit

Write a description of your method and all relevant aspects of your experiments. Write it in line in the text under the relevant section. Your writing should be similar to a blog post so that anyone in the class can fully understand the pain/joy/content you experienced (feel free to take copious input from an LLM but do not substitute an LLM for learning).

The total length should be no less than 500 words and should normally exceed 800 words. Your ultimate aim in writing your answers to mention important aspect of your assignment for the task, with emphasis on the last task. This will include the methods employed listing all the software tools you employed, as well experimental aspects such as when you took the picture, how many pictures you took, whether the method works in dawn, morning, and dusk, and so on. Be creative.

Do not submit any large file such as model weights and ancillary files that we can easily get from the internet to reproduce.

A short blog: measuring a campus avenue from one photo.

When I first read the title (“Who moved my bicycle?”) I thought the hard part would be the deep learning. It wasn’t. Detecting the bicycle took one call to a pretrained model. The real work, and the fun, was in the geometry: convincing myself that you can recover a real distance in metres from a flat photograph, with no depth sensor, no second camera, just one image and one known length.

The reason it works is the cross ratio. In a photograph, perspective ruins ordinary measurement – equal real distances do not stay equal on the image, which is obvious the moment you look at how the avenue’s edges squeeze together towards the arch. So “count the pixels and scale” is hopeless. But there is exactly one quantity built from four points on a line that survives projection unchanged: the cross ratio. That is the whole trick. If I can find four collinear points and I know three of the real distances between them, the fourth falls out. This is the same idea as the surveillance-camera example from the 25-May class, where the train gets located between two stations using the point where the rails converge.

My four points, in order away from the camera, were A' the Main Building red line, B' the bicycle, C' the intersection at the third pole, and V' the vanishing point – which, conveniently, sits almost exactly at the arch. The first thing I learned the hard way is that the arch and the vanishing point are essentially the same point in this photo, so I could not use “MB, arch, vanishing point” as my three references; the arch and V' were too close in pixels and the arithmetic became unstable. Switching to the intersection (well separated from the arch in the image) as the middle reference fixed that. A second neat realisation: the cross ratio doesn’t care how you put coordinates on the line, so I only needed the pixel row of each point, not both coordinates. That cut the measurement effort in half and made the dark blue annotation line easy to draw and read.

Tools. For detection I used Ultralytics YOLOv8, with the COCO class ids (bicycle = 1, person = 0). For pixel coordinates I used MS Paint and a short Pillow script; for the ground-truth distance, Google Maps’ measure tool gave 160.35 m from MB to the arch, which I rounded to 160 m. The geometry itself is a few lines of Python. I cross-checked the bicycle result two ways – once using the vanishing point as the point at infinity (124 m), once using three finite landmarks with no vanishing point (121 m) – and they agreed to within a few metres, which gave me confidence the method, and my reading of the points, were sound. I report the bicycle as roughly 120 m from the arch.

The part that surprised me most was how much the choice of point on the detected object matters. The YOLO box is a rectangle, but the bicycle only touches the ground at the bottom. When I tried the box centre or the saddle instead of the wheel contact, the answer moved by more than twenty

metres. That is not a rounding issue – it is geometry: near the vanishing point the image is so compressed that one pixel is worth about 0.16 m, so where exactly you “stand” the object changes a lot. The lesson is that the reference point must be the ground-contact point, every time. By contrast, rounding the pixel rows to the nearest 10 barely moved the result, so the human pixel-reading was not the bottleneck; the modelling choice was.

For the second part I repeated the whole pipeline on my own photographs in my housing society, on the walkway outside building B-6. I took a first-person view with another person standing on the receding walkway, and a third-person view showing me in the act of photographing them so both of us are in frame. Both shots were taken at dusk, around 7:25 PM, and that mattered: in the fading light the far end of the walkway and my person’s feet were noticeably harder to pin down than the bright daytime campus shot in Part 1, which widened my error on the vanishing point. Taking the depth of the walkway as about 100 m, the cross ratio placed the person roughly 7–8 m from the camera, and the third-person photo – where the two of us stand about 7–8 m apart – agreed, which was satisfying to see. The same two scripts handled detection and distance; the only change was keeping the **person** class instead of **bicycle**.

If I had to summarise the experience: the deep learning is a convenience, not the substance. The substance is one old, elegant projective invariant, and the discipline of choosing the right point and the right references. Most of my errors came not from the maths but from the assumptions underneath it – a not-quite-straight path edge, an object slightly off the line, a tilted phone. Respect those assumptions and a single photo really does give you metres.